

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_a6bedd49-683\agent-a903a4cb64874fed3.jsonl`
- SHA-256: `bfc4aba5828afae2ebdaf7e95281b2bc9b5d00c811a968baac29a17b8377e10a`
- Source modified: `2026-07-06T23:51:35+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_a6bedd49-683`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T23:48:46.053Z)

Reviewing GitHub PR #55 "F6: persist policy decision traces" (branch feat/f6-policy-decisions -> main). ADR 0012 F6.
+387/-11, 7 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr55 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f6-policy-decisions:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr55/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F6 DOES:

- db.py: new policy_decisions table in BOTH _SCHEMA and @_migration(5)
_migration_005_policy_decisions (both CREATE TABLE IF NOT EXISTS).
Schema: (id PK, content_item_id FK content_items, policy_id FK policies, passed INT, trace_json TEXT, evaluated_at TEXT, UNIQUE(content_item_id, policy_id)).
Repos: record_policy_decision (UPSERT on (content_item_id,policy_id):
passed/trace_json/evaluated_at), get_policy_decision, list_policy_decisions_for_content.
_policy_trace_json = json.dumps(trace, sort_keys, compact). _row_to_policy_decision json.loads + validates list.
- models.py: PolicyDecision (frozen). policy.py: stage_results_to_trace(results) -> [{stage_name,passed,reason,metadata}].
- telethon_client.py: _evaluate_policy_stages now returns (passed, results, policy_id).
_record_policy_decision(inserted, policy_id, policy_passed, policy_results):
guards 'if not policy_results or policy_id is None or inserted.id is None: return'; else
db.record_policy_decision(content_item_id=inserted.id, ...).
Called in on_new_message AND backfill AFTER insert_video (so the FK content_item exists).
on_new_message/backfill are '# pragma: no cover'.

VERIFIED by main reviewer: gate green (551 passed @ 94.34%); migration idempotent + reaches user_version=5 on a legacy v1 DB; a full pipeline trace serializes + persists; UPSERT is idempotent (1 row on re-record); policy decisions are stored SEPARATELY from delivery_attempts (addresses an earlier Track C finding).
Note deviations/limits already spotted: schema is one-row-per-decision-with-JSON-trace (ADR literally proposed per-stage rows); NO TTL/cleanup (ADR open question); metadata-rejected (Tier-1 FilterEngine) items get NO policy_decision row (only the evaluate_pipeline Tier 2-4 trace is recorded).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.
Severity: high (migration data loss/corruption or a write bug that crashes ingestion/persists wrong data), medium (real latent bug/FK/serialization crash), low (note/deviation/growth).

DIMENSION: migration 005 + schema correctness.
Investigate: idempotency (fresh DB double-init; legacy v1/v2 DB; re-run) ? does _apply_schema's drop-empty-legacy logic interact with policy_decisions? Is
policy_decisions created on BOTH fresh (_SCHEMA) and migrated (mig 005) paths consistently (same

columns/constraints)? FK enforcement (PRAGMA foreign_keys=ON):
can policy_decisions ever reference a missing content_item/policy? UNIQUE(content_item_id, policy_id) correct for idempotency? Does migration 005 run in the right order (after 003/004 which create content_items/policies it FKs to)? On a legacy DB where content_items/policies exist post-003/004, does the FK hold?
user_version reaches 5? Any issue if policy_decisions already exists (partial prior run)? Run legacy + fresh repros. Report findings.

2. assistant (2026-07-06T23:48:48.427Z)

I'll review PR #55's migration 005 + schema correctness. Let me start by reading the relevant code.

3. user (2026-07-06T23:48:50.710Z)

```
42b5637 feat(policy): persist policy decision traces
da0ab47 feat(policy): add deep inspection budget boundary (#54)
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
6d3d7a6 feat(policy): add F2 topic preferences (#51)
---
tg_video_filter
```

4. user (2026-07-06T23:48:51.745Z)

```
src/tg_video_filter/db.py
src/tg_video_filter/models.py
src/tg_video_filter/policy.py
---
2007 src/tg_video_filter/db.py
```

5. assistant (2026-07-06T23:48:53.833Z)

Let me read the core files.

6. user (2026-07-06T23:48:55.896Z)

```
42:_SCHEMA = """
186:CREATE TABLE IF NOT EXISTS policy_decisions (
198:# migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
200:# migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
208:    then runs any pending migrations via the ``user_version`` ladder.
211:    await _apply_schema(conn)
214:async def _apply_schema(conn: aiosqlite.Connection) -> None:
218:    await conn.execute("PRAGMA foreign_keys = ON")
220:    await conn.executescript(_SCHEMA)
221:    # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
237:# Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
238:# Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
257:    """Register a migration that brings the DB up to ``user_version == version``."""
331:    """Run every registered migration whose version > current ``user_version``."""
332:    async with conn.execute("PRAGMA user_version") as cur:
340:        await conn.execute(f"PRAGMA user_version = {version}")
392:    # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
414:    # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
525:    DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
532:    # already run (or the DB is fresh and _SCHEMA's legacy tables were already
533:    # dropped by _apply_schema post-003). Nothing to backfill.
648:    # _SCHEMA creates it; on migrated DBs this is the first time.
778:@migration(5)
779:async def _migration_005_policy_decisions(conn: aiosqlite.Connection) -> None:
783:    CREATE TABLE IF NOT EXISTS policy_decisions (
925:def _row_to_policy_decision(row: aiosqlite.Row) -> PolicyDecision:
928:    raise ValueError("policy_decisions.trace_json must decode to a JSON array")
```

```

1396:async def record_policy_decision(
1409:     """INSERT INTO policy_decisions
1425:     decision = await get_policy_decision(conn, content_item_id, policy_id)
1430:async def get_policy_decision(
1438:     """SELECT * FROM policy_decisions
1443:     return _row_to_policy_decision(row) if row else None
1446:async def list_policy_decisions_for_content(
1453:     """SELECT * FROM policy_decisions
1459:     return [_row_to_policy_decision(row) for row in rows]
1980:     "record_policy_decision",
1981:     "get_policy_decision",
1982:     "list_policy_decisions_for_content",

```

7. user (2026-07-06T23:48:59.000Z)

```

42     _SCHEMA = ""
43     CREATE TABLE IF NOT EXISTS accounts (
44         id            INTEGER PRIMARY KEY,
45         telegram_id   INTEGER UNIQUE,
46         provider      TEXT NOT NULL DEFAULT 'telegram',
47         display_name  TEXT,
48         created_at    TEXT NOT NULL
49     );
50
51     CREATE TABLE IF NOT EXISTS telegram_sessions (
52         account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
53         phone         TEXT,
54         session_path  TEXT NOT NULL,
55         enabled       INTEGER NOT NULL DEFAULT 1,
56         created_at    TEXT NOT NULL,
57         last_login_at TEXT
58     );
59
60     CREATE TABLE IF NOT EXISTS channels (
61         id            INTEGER PRIMARY KEY AUTOINCREMENT,
62         user_id       INTEGER NOT NULL REFERENCES accounts(id),
63         channel_id    INTEGER NOT NULL,
64         title         TEXT,
65         seen_first_at TEXT NOT NULL,
66         UNIQUE (user_id, channel_id)
67     );
68
69     CREATE TABLE IF NOT EXISTS videos (
70         id            INTEGER PRIMARY KEY AUTOINCREMENT,
71         user_id       INTEGER NOT NULL REFERENCES accounts(id),
72         channel_id    INTEGER NOT NULL,
73         message_id    INTEGER NOT NULL,
74         document_id   INTEGER,
75         duration_s    INTEGER,
76         width         INTEGER,
77         height        INTEGER,
78         size_bytes    INTEGER,
79         mime_type     TEXT,
80         matched       INTEGER NOT NULL DEFAULT 0,
81         forwarded     INTEGER NOT NULL DEFAULT 0,
82         seen_at       TEXT NOT NULL,
83         UNIQUE (user_id, channel_id, message_id)
84     );
85
86     CREATE TABLE IF NOT EXISTS filter_config (
87         user_id       INTEGER PRIMARY KEY REFERENCES accounts(id),
88         config_json   TEXT NOT NULL
89     );
90
91     -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,

```

```

92 -- ``content_items``, and ``policies`` are populated by migration 003 (and by
93 -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
94 -- are created empty here so the schema is complete ? Track C (#28) implements
95 -- the delivery logic that populates them.
96
97 CREATE TABLE IF NOT EXISTS sources (
98     id                INTEGER PRIMARY KEY AUTOINCREMENT,
99     account_id        INTEGER NOT NULL REFERENCES accounts(id),
100    provider           TEXT NOT NULL DEFAULT 'telegram',
101    provider_source_id TEXT NOT NULL,
102    title              TEXT,
103    kind               TEXT NOT NULL DEFAULT 'channel',
104    watched            INTEGER NOT NULL DEFAULT 0,
105    seen_first_at      TEXT NOT NULL,
106    last_seen_at       TEXT,
107    UNIQUE (account_id, provider, provider_source_id)
108 );
109
110 CREATE TABLE IF NOT EXISTS content_items (
111     id                INTEGER PRIMARY KEY AUTOINCREMENT,
112     account_id        INTEGER NOT NULL REFERENCES accounts(id),
113     source_id         INTEGER NOT NULL REFERENCES sources(id),
114     provider          TEXT NOT NULL DEFAULT 'telegram',
115     provider_message_id TEXT NOT NULL,
116     content_type       TEXT NOT NULL DEFAULT 'video',
117     text              TEXT,
118     link_url          TEXT,
119     media_ref         TEXT,
120     document_id       TEXT,
121     duration_s        INTEGER,
122     width             INTEGER,
123     height            INTEGER,
124     size_bytes        INTEGER,
125     mime_type         TEXT,
126     dedup_key         TEXT NOT NULL,
127     matched           INTEGER NOT NULL DEFAULT 0,
128     seen_at           TEXT NOT NULL,
129     UNIQUE (account_id, source_id, provider_message_id),
130     UNIQUE (account_id, dedup_key)
131 );
132
133 CREATE TABLE IF NOT EXISTS policies (
134     id                INTEGER PRIMARY KEY AUTOINCREMENT,
135     account_id        INTEGER NOT NULL REFERENCES accounts(id),
136     name              TEXT NOT NULL,
137     config_json       TEXT NOT NULL,
138     is_default        INTEGER NOT NULL DEFAULT 0,
139     created_at        TEXT NOT NULL,
140     updated_at        TEXT NOT NULL,
141     UNIQUE (account_id, name)
142 );
143
144 -- ADR 0011: exactly one default policy per account (structural invariant).
145 CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
146 ON policies(account_id) WHERE is_default = 1;
147
148 CREATE TABLE IF NOT EXISTS destinations (
149     id                INTEGER PRIMARY KEY AUTOINCREMENT,
150     account_id        INTEGER NOT NULL REFERENCES accounts(id),
151     provider          TEXT NOT NULL DEFAULT 'telegram',
152     target            TEXT NOT NULL,
153     kind              TEXT NOT NULL DEFAULT 'saved',
154     title             TEXT,
155     is_output         INTEGER NOT NULL DEFAULT 0,
156     created_at        TEXT NOT NULL,

```

```

157         UNIQUE (account_id, provider, target)
158     );
159
160     CREATE TABLE IF NOT EXISTS routing_rules (
161         id                INTEGER PRIMARY KEY AUTOINCREMENT,
162         account_id        INTEGER NOT NULL REFERENCES accounts(id),
163         source_id         INTEGER NOT NULL REFERENCES sources(id),
164         policy_id         INTEGER REFERENCES policies(id),
165         destination_id    INTEGER NOT NULL REFERENCES destinations(id),
166         enabled           INTEGER NOT NULL DEFAULT 1,
167         priority          INTEGER NOT NULL DEFAULT 0,
168         created_at        TEXT NOT NULL,
169         UNIQUE (account_id, source_id, destination_id)
170     );
171
172     CREATE TABLE IF NOT EXISTS delivery_attempts (
173         id                INTEGER PRIMARY KEY AUTOINCREMENT,
174         content_item_id    INTEGER NOT NULL REFERENCES content_items(id),
175         destination_id    INTEGER NOT NULL REFERENCES destinations(id),
176         route_id          INTEGER REFERENCES routing_rules(id),
177         status            TEXT NOT NULL DEFAULT 'pending',
178         attempt_count     INTEGER NOT NULL DEFAULT 0,
179         last_error        TEXT,
180         created_at        TEXT NOT NULL,
181         delivered_at      TEXT,
182         next_retry_at     TEXT,
183         UNIQUE (content_item_id, destination_id)
184     );
185
186     CREATE TABLE IF NOT EXISTS policy_decisions (
187         id                INTEGER PRIMARY KEY AUTOINCREMENT,
188         content_item_id    INTEGER NOT NULL REFERENCES content_items(id),
189         policy_id         INTEGER NOT NULL REFERENCES policies(id),
190         passed            INTEGER NOT NULL,
191         trace_json        TEXT NOT NULL,
192         evaluated_at      TEXT NOT NULL,
193         UNIQUE (content_item_id, policy_id)
194     );
195     """
196
197     # Note: the ``idx_videos_user_document`` partial unique index is created by
198     # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
199     # which is present on fresh tables but only added to legacy v1 DBs by the
200     # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
201     # DBs that don't yet have the column.

```

8. assistant (2026-07-06T23:49:00.860Z)

Now let me read the migration 005 and _apply_schema logic.

9. user (2026-07-06T23:49:02.173Z)

```

205     """Open the database at ``db_path`` and create/migrate the schema.
206
207     Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
208     then runs any pending migrations via the ``user_version`` ladder.
209     """
210     async with aiosqlite.connect(db_path) as conn:
211         await _apply_schema(conn)
212
213
214     async def _apply_schema(conn: aiosqlite.Connection) -> None:
215         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
216         # and a busy timeout so concurrent access waits instead of failing immediately.
217         await conn.execute("PRAGMA journal_mode = WAL")

```

```

218     await conn.execute("PRAGMA foreign_keys = ON")
219     await conn.execute("PRAGMA busy_timeout = 5000")
220     await conn.executescript(_SCHEMA)
221     # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
222     # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
223     # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
224     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
225     # any *empty* recreations of those legacy tables from a subsequent
226     # ``init_db`` are dropped here so they don't shadow the renamed copies.
227     if await _table_exists(conn, "channels_deprecated"):
228         await conn.execute("DROP TABLE IF EXISTS channels")
229         await conn.execute("DROP TABLE IF EXISTS videos")
230         await conn.execute("DROP TABLE IF EXISTS filter_config")
231     await _apply_migrations(conn)
232     await conn.commit()
233
234
235     # --- migrations -----
236     #
237     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
238     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
239     # the source of truth for *fresh* databases; the migrations below reconcile
240     # *existing* databases that were created by an older schema version.
241     #
242     # To add a new migration:
243     # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
244     # 2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
245     # 3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
246     #    ``init_db``, and asserts the new structure.
247
248     _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
249
250
251     def _migration(
252         version: int,
253     ) -> Callable[
254         [Callable[[aiosqlite.Connection], Awaitable[None]]],
255         Callable[[aiosqlite.Connection], Awaitable[None]],
256     ]:
257         """Register a migration that brings the DB up to ``user_version == version``."""
258
259         def decorator(
260             fn: Callable[[aiosqlite.Connection], Awaitable[None]],
261         ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
262             _MIGRATIONS.append((version, fn))
263             return fn
264
265         return decorator
266
267
268     # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
269     #
270     # Pure function: computes the ``content_items.dedup_key`` value from metadata
271     # already extracted at ingestion time (no download ? honors ADR 0003). The
272     # decision tree is ordered strongest-key first; each branch is guarded so
273     # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0``).
274     # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
275
276
277     def compute_dedup_key(
278         *,
279         provider: str,
280         document_id: str | None = None,
281         source_id: int | str | None = None,
282         provider_message_id: str | None = None,

```

```

283     size_bytes: int | None = None,
284     duration_s: int | None = None,
285     height: int | None = None,
286     width: int | None = None,
287 ) -> str:
288     """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
289
290     Implements the ADR 0011 Option 1A decision tree:
291
292     - **Telegram** (``provider == "telegram"``):
293       1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
294       2. Else if **all four** fingerprint fields (size, duration, height, width)
295          are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
296       3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
297          ``source_id`` is required because Telegram ``message_id`` is only
298          unique within a channel).
299     - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
300       ``"{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
301       providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
302
303     Raises ``ValueError`` if the fallback branch is reached without the fields
304     it requires (``source_id`` + ``provider_message_id``) ? a degenerate
305     ``dedup_key`` is worse than a loud failure at ingestion time.
306     """
307     p = provider.lower()
308     if p == "telegram":
309         if document_id is not None and document_id != "":
310             return f"tg:doc:{document_id}"
311         # All four fingerprint fields are required to avoid a degenerate
312         # ``tg:fp:0:0:0:0`` key when some metadata is missing.
313         if None not in (size_bytes, duration_s, height, width):
314             return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
315         # Tier-0 fallback.
316         if source_id is None or provider_message_id is None:
317             raise ValueError(
318                 "Telegram dedup_key fallback requires source_id and provider_message_id"
319             )
320         return f"tg:msg:{source_id}:{provider_message_id}"
321     # Future-proof default for providers without a dedicated branch.
322     if source_id is None or provider_message_id is None:
323         raise ValueError(
324             f"dedup_key for provider={provider!r} requires source_id and"
325             f"provider_message_id"
326         )
327     return f"{p}:msg:{source_id}:{provider_message_id}"
328
329
330 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
331     """Run every registered migration whose version > current ``user_version``."""
332     async with conn.execute("PRAGMA user_version") as cur:
333         row = await cur.fetchone()
334         current = int(row[0]) if row else 0
335
336     for version, fn in _MIGRATIONS:
337         if version <= current:
338             continue
339         await fn(conn)
340         await conn.execute(f"PRAGMA user_version = {version}")
341
342
343 async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
344     """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
345     async with conn.execute(f"PRAGMA table_info({table})") as cur:

```


12. assistant (2026-07-06T23:49:11.370Z)

Migration 005 schema matches `_SCHEMA` exactly. Now the critical question: FK enforcement. `_apply_schema` runs executescript(_SCHEMA)` FIRST (creating policy_decisions with FK to content_items and policies), then _apply_migrations`. On a legacy DB, does content_items/policies exist at the time _SCHEMA runs? Let me check migration ordering and the content_items/policies creation path.`

13. user (2026-07-06T23:49:14.184Z)

```
110:CREATE TABLE IF NOT EXISTS content_items (  
133:CREATE TABLE IF NOT EXISTS policies (  
262:     _MIGRATIONS.append((version, fn))  
350:@_migration(1)  
358:     await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")  
367:@_migration(2)  
442:     # ALTER TABLE RENAME rewrites FK references in *other* tables to follow  
497:     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")  
516:     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")  
519:@_migration(3)  
677:     await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")  
678:     await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")  
679:     await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")  
682:@_migration(4)  
778:@_migration(5)
```

14. assistant (2026-07-06T23:49:19.505Z)

`content_items` and `policies` are in `_SCHEMA` (created fresh via ``IF NOT EXISTS``), so on both fresh and legacy paths they exist before migration 005 runs. Since `_SCHEMA` creates `policy_decisions` too, and `content_items/policies` are also in `_SCHEMA`, the FK targets always exist.

Now the key FK concern: ``REFERENCES content_items(id)`` and ``REFERENCES policies(id)``. FK checks in SQLite are deferred at CREATE time (the referenced table just needs to exist). Let me check migration 002 ? it does an FK rebuild that RENAMES tables, and the comment at line 442 says "ALTER TABLE RENAME rewrites FK references in other tables". This is the danger: if `policy_decisions` exists in `_SCHEMA` and migration 002 renames a table, could SQLite rewrite `policy_decisions'` FKs? Let me examine migration 002 closely.

15. user (2026-07-06T23:49:20.492Z)

```
367     @_migration(2)  
368     async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:  
369         """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.  
370  
371         ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the  
372         data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,  
373         so their integer ``user_id`` values still resolve to the same identity (now  
374         via ``accounts.id``). The legacy ``users`` table is renamed to  
375         ``users_deprecated`` and kept for one release as a rollback safety net; a  
376         follow-up migration drops it.  
377  
378         FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with  
379         ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape  
380         as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting  
381         those FKs to point at ``users_deprecated``, diverging from fresh installs  
382         and breaking inserts for new accounts ? see PR #33 review.)  
383  
384         Idempotent: a no-op once the legacy ``users`` table has been renamed.  
385         ""  
386         # If the legacy users table is gone, this migration has already run  
387         # (or the DB is fresh and only has accounts). Nothing to do.  
388         if not await _table_exists(conn, "users"):  
389             return
```

```

390
391 # Build accounts from the legacy users table. accounts.id := users.id.
392 # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
393 # and recreated here so the INSERT can populate it cleanly.)
394 await conn.execute("DROP TABLE IF EXISTS accounts")
395 await conn.execute(
396     """
397     CREATE TABLE accounts (
398         id            INTEGER PRIMARY KEY,
399         telegram_id   INTEGER UNIQUE,
400         provider      TEXT NOT NULL DEFAULT 'telegram',
401         display_name  TEXT,
402         created_at    TEXT NOT NULL
403     )
404     """
405 )
406 await conn.execute(
407     """
408     INSERT INTO accounts (id, telegram_id, provider, created_at)
409     SELECT id, id, 'telegram', created_at FROM users
410     """
411 )
412
413 # Build telegram_sessions, one row per legacy userbot user.
414 # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
415 await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
416 await conn.execute(
417     """
418     CREATE TABLE telegram_sessions (
419         account_id   INTEGER PRIMARY KEY REFERENCES accounts(id),
420         phone        TEXT,
421         session_path TEXT NOT NULL,
422         enabled      INTEGER NOT NULL DEFAULT 1,
423         created_at   TEXT NOT NULL,
424         last_login_at TEXT
425     )
426     """
427 )
428 # session_path is built from the *telegram_id* (= old users.id) to match
429 # the convention telethon_client.build_client used pre-migration. The
430 # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
431 # so this migration does not need to know the runtime path.
432 await conn.execute(
433     """
434     INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
435     created_at)
436     SELECT id, phone, 'user_' || id || '.session', enabled, created_at
437     FROM users
438     """
439 )
440
441 # Rebuild channels / videos / filter_config so their FK constraints point
442 # at accounts(id) instead of the legacy users(id). SQLite's
443 # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
444 # the rename ? so a naive rename of users ? users_deprecated would leave
445 # these tables referencing users_deprecated(id), diverging from fresh DBs
446 # (which reference accounts(id)) and breaking inserts for new accounts.
447 # The rebuild: rename the data-holding legacy table out of the way, create
448 # the new one with the correct FK, copy the data back, drop the renamed copy.
449 for table, new_ddl, copy_cols in (
450     (
451         "channels",
452         """
453         CREATE TABLE channels (
454             id
455             INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

454         user_id      INTEGER NOT NULL REFERENCES accounts(id),
455         channel_id    INTEGER NOT NULL,
456         title         TEXT,
457         seen_first_at TEXT NOT NULL,
458         UNIQUE (user_id, channel_id)
459     )
460     """
461     "id, user_id, channel_id, title, seen_first_at",
462 ),
463 (
464     "videos",
465     """
466     CREATE TABLE videos (
467         id            INTEGER PRIMARY KEY AUTOINCREMENT,
468         user_id       INTEGER NOT NULL REFERENCES accounts(id),
469         channel_id    INTEGER NOT NULL,
470         message_id    INTEGER NOT NULL,
471         document_id   INTEGER,
472         duration_s    INTEGER,
473         width         INTEGER,
474         height        INTEGER,
475         size_bytes    INTEGER,
476         mime_type     TEXT,
477         matched       INTEGER NOT NULL DEFAULT 0,
478         forwarded     INTEGER NOT NULL DEFAULT 0,
479         seen_at       TEXT NOT NULL,
480         UNIQUE (user_id, channel_id, message_id)
481     )
482     """
483     "id, user_id, channel_id, message_id, document_id, duration_s, width, "
484     "height, size_bytes, mime_type, matched, forwarded, seen_at",
485 ),
486 (
487     "filter_config",
488     """
489     CREATE TABLE filter_config (
490         user_id       INTEGER PRIMARY KEY REFERENCES accounts(id),
491         config_json   TEXT NOT NULL
492     )
493     """
494     "user_id, config_json",
495 ),
496 ):
497     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
498     await conn.execute(new_ddl)
499     await conn.execute(
500         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
501     )
502     await conn.execute(f"DROP TABLE {table}__legacy_data")
503
504     # The partial unique index for document_id dedup (created by migration 001
505     # against the legacy videos table) was dropped along with the legacy copy
506     # above. Recreate it on the new videos table.
507     await conn.execute(
508         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
509         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
510     )
511
512     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
513     # net for one release per ADR 0010). The data-holding channels/videos/
514     # filter_config tables were already rebuilt above with FKs to accounts(id),
515     # so renaming users no longer retargets any live FK.
516     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
517

```

```

518
519     @_migration(3)
520     async def _migration_003_domain_schema(conn:aiosqlite.Connection) -> None:
521         """Land the provider-neutral domain tables (ADR 0011, Track B).
522
523         Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
524         ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
525         DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
526         them), then backfills the three populated ones from the legacy

```

16. assistant (2026-07-06T23:49:26.853Z)

This is the critical interaction. On a **legacy v1/v2 DB**, `_apply_schema` runs ``executescript(_SCHEMA)`` first ? which creates ``policy_decisions`` with FKs to ``content_items(id)`` and ``policies(id)``. Those tables are also in `_SCHEMA`, so they exist. So far fine.

But the real question is migration 002's table-rename FK-rewrite behavior. SQLite's ``ALTER TABLE ... RENAME TO`` rewrites FK references in *other* tables that point to the renamed table. Migration 002 renames ``channels`/`videos`/`filter_config`` (via ``__legacy_data``) and ``users``. ``policy_decisions`` doesn't reference any of those, so it's unaffected. Good.

Let me now actually run the repros ? the most reliable way to catch FK/ordering/idempotency bugs. Let me check for existing test helpers to build a legacy DB.

17. user (2026-07-06T23:49:28.113Z)

```

526         them), then backfills the three populated ones from the legacy
527         ``channels`` / ``videos`` / ``filter_config`` tables and renames the
528         legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
529         pattern). Idempotent: a no-op once the legacy tables have been renamed.
530         """
531         # Idempotency gate: if the legacy videos table is gone, this migration has
532         # already run (or the DB is fresh and _SCHEMA's legacy tables were already
533         # dropped by _apply_schema post-003). Nothing to backfill.
534         if not await _table_exists(conn, "videos"):
535             return
536
537         # 1a. Backfill sources from channels first (known title + watched=1). Order
538         #     matters: channels-first so title/watched data wins; the videos-based
539         #     backfill below fills gaps for orphan channels (review finding #1).
540         await conn.execute(
541             """
542             INSERT OR IGNORE INTO sources
543                 (account_id, provider, provider_source_id, title, kind, watched,
544                  seen_first_at, last_seen_at)
545             SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
546                    1, seen_first_at, seen_first_at
547             FROM channels
548             """
549         )
550
551         # 1b. Backfill sources from videos for channels that have no channels row
552         #     (e.g. backfill-only channels that were never seen via live traffic).
553         #     INSERT OR IGNORE is a no-op on (account, provider, source_id) conflicts
554         #     ? the channels-based row above survives with its title + watched=1.
555         await conn.execute(
556             """
557             INSERT OR IGNORE INTO sources
558                 (account_id, provider, provider_source_id, kind, watched, seen_first_at)
559             SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
560                    'channel', 0, MIN(v.seen_at)
561             FROM videos v
562             GROUP BY v.user_id, v.channel_id
563             """

```

```

564 )
565
566 # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
567 #   Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
568 #   (no silent drops, even on fingerprint collisions).
569 #   Phase B ? promote to tg:doc: for items with a document_id (one per doc-id
group).
570 #   Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint
group).
571 #   The non-promoted losers stay at tg:msg: (unique) ? no silent data loss.
572 await conn.execute(
573     """
574     INSERT INTO content_items
575         (account_id, source_id, provider, provider_message_id, content_type,
576          document_id, duration_s, width, height, size_bytes, mime_type,
577          dedup_key, matched, seen_at)
578     SELECT
579         v.user_id,
580         s.id,
581         'telegram',
582         CAST(v.message_id AS TEXT),
583         'video',
584         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
585         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
586         'tg:msg:' || s.id || ':' || v.message_id, -- Tier-0 (always unique)
587         v.matched,
588         v.seen_at
589     FROM videos v
590     JOIN sources s
591         ON s.account_id = v.user_id
592         AND s.provider = 'telegram'
593         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
594     """
595 )
596
597 # Phase B: promote one row per (account, document_id) to tg:doc:.
598 await conn.execute(
599     """
600     UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
601     WHERE document_id IS NOT NULL
602         AND dedup_key LIKE 'tg:msg:%'
603         AND id IN (
604             SELECT MIN(id) FROM content_items
605             WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
606             GROUP BY account_id, document_id
607         )
608     """
609 )
610
611 # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
612 await conn.execute(
613     """
614     UPDATE content_items SET dedup_key =
615         'tg:fp:' || size_bytes || ':' || duration_s || ':'
616         || height || ':' || width
617     WHERE document_id IS NULL
618         AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
619         AND height IS NOT NULL AND width IS NOT NULL
620         AND dedup_key LIKE 'tg:msg:%'
621         AND id IN (
622             SELECT MIN(id) FROM content_items
623             WHERE document_id IS NULL
624                 AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
625                 AND height IS NOT NULL AND width IS NOT NULL
626                 AND dedup_key LIKE 'tg:msg:%'

```

```

627         GROUP BY account_id, size_bytes, duration_s, height, width
628     )
629     """
630 )
631
632 # 3. Backfill policies from filter_config: one 'default' policy per account
633 #     (Option 2A). is_default=1 so the legacy load_filter_config shim reads
634 #     this row. UNIQUE(account_id, name) dedups idempotent re-runs.
635 await conn.execute(
636     """
637     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
638                                     created_at, updated_at)
639     SELECT user_id, 'default', config_json, 1,
640            strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
641            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
642     FROM filter_config
643     """
644 )
645
646 # 4. Create the policies default-partial index (sig after backfill so the
647 #     UNIQUE constraint only applies once rows are populated). On fresh DBs
648 #     _SCHEMA creates it; on migrated DBs this is the first time.
649 await conn.execute(
650     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
651     "ON policies(account_id) WHERE is_default = 1"
652 )
653
654 # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
655 #     produce a content_items row. A mismatch signals fingerprint collisions
656 #     where two distinct videos shared (size,dur,h,w) ? the losers stay at
657 #     tg:msg: keys and are still present. Log a warning so the operator can
658 #     investigate; the data is preserved in *_deprecated tables.
659 async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
660     vc = (await cur.fetchone())[0]
661 async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
662     cic = (await cur.fetchone())[0]
663 if vc != cic:
664     import logging
665
666     _logger = logging.getLogger(__name__)
667     _logger.warning(
668         "migration_003: %d videos rows produced %d content_items rows "
669         "%d gap). Colliding items remain in videos_deprecated.",
670         vc,
671         cic,
672         vc - cic,
673     )
674
675 # 6. Rename (don't drop) the legacy tables ? kept for one release as a
676 #     rollback safety net (ADR 0010 pattern).
677 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
678 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
679 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
680
681
682 @_migration(4)
683 async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
684     """Backfill default destinations and delivery state (ADR 0011, Track C).
685

```

18. assistant (2026-07-06T23:49:31.528Z)

Now I understand the full picture. Let me run repros to verify the migration paths concretely. Let me build test scripts for: (1) fresh DB double-init, (2) legacy v1 DB migration, (3) FK enforcement on policy_decisions.

19. user (2026-07-06T23:49:33.992Z)

```
103:     enabled = await db.list_enabled_users(conn)
198:     # And it is excluded from the enabled-users list.
199:     assert await db.list_enabled_users(conn) == []
689:async def test_fresh_db_user_version_is_latest(conn: aiosqlite.Connection) -> None:
690:     """init_db/_apply_schema must bump user_version to the highest migration."""
692:     async with conn.execute("PRAGMA user_version") as cur:
707:         """A v1 DB (no document_id, user_version=0) must be migrated on open."""
710:         # Build a legacy v1 schema WITH the real FK definitions (REFERENCES users),
711:         # no document_id column, user_version = 0. Using the real FK constraints
713:         # FK references to follow a rename (users ? users_deprecated) instead of
716:         CREATE TABLE users (
722:             user_id INTEGER NOT NULL REFERENCES users(id),
728:             user_id INTEGER NOT NULL REFERENCES users(id),
736:             user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
743:             "INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
'2024-01-01T00:00:00Z')"
756:         # user_version is 0 on a freshly created DB.
765:         # Identity split (migration 002): users renamed, accounts present.
769:         assert not await db._table_exists(conn, "users")
770:         # accounts.id copied from users.id (integer-preserving).
845:         # user_version bumped to the highest registered migration.
===
__pycache__
conftest.py
test_backfill.py
test_bootstrap.py
test_command_service.py
test_commands.py
test_config.py
test_db.py
test_delivery.py
test_filter_engine.py
test_filters_content_type.py
test_filters_duration.py
test_filters_resolution.py
test_filters_size.py
test_main.py
test_manager.py
test_policy.py
test_telethon_extraction.py
```

20. assistant (2026-07-06T23:49:36.379Z)

Let me read the existing legacy v1 test to reuse its seeding, then write a targeted repro that exercises the full legacy migration + FK enforcement on policy_decisions.

21. user (2026-07-06T23:49:37.379Z)

```
705
706     async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
707         """A v1 DB (no document_id, user_version=0) must be migrated on open."""
708         db_path = str(tmp_path) + "/legacy.db" # type: ignore[operator]
709
710         # Build a legacy v1 schema WITH the real FK definitions (REFERENCES users),
711         # no document_id column, user_version = 0. Using the real FK constraints
712         # is what lets this test catch the PR #33 regression where SQLite rewrites
713         # FK references to follow a rename (users ? users_deprecated) instead of
714         # retargeting them to accounts.
715         legacy_schema = """
716         CREATE TABLE users (
717             id INTEGER PRIMARY KEY, phone TEXT,
718             enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
719         );
```

```

720 CREATE TABLE channels (
721     id INTEGER PRIMARY KEY AUTOINCREMENT,
722     user_id INTEGER NOT NULL REFERENCES users(id),
723     channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
724     UNIQUE (user_id, channel_id)
725 );
726 CREATE TABLE videos (
727     id INTEGER PRIMARY KEY AUTOINCREMENT,
728     user_id INTEGER NOT NULL REFERENCES users(id),
729     channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
730     duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER,
731     mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
732     forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
733     UNIQUE (user_id, channel_id, message_id)
734 );
735 CREATE TABLE filter_config (
736     user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
737 );
738 """
739 async with aiosqlite.connect(db_path) as conn:
740     await conn.executescript(legacy_schema)
741     # Seed rows so we can confirm data survives the migration.
742     await conn.execute(
743         "INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
744 '2024-01-01T00:00:00Z')"
745     )
746     await conn.execute(
747         "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
748         "VALUES (111, -1001, 'pre-migration', '2024-01-01T00:00:00Z')"
749     )
750     await conn.execute(
751         "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
752 height, "
753         "size_bytes, mime_type, matched, forwarded, seen_at) "
754         "VALUES (111, -1001, 1, 60, 1920, 1080, 1000000, 'video/mp4', 1, 1,
755 '2024-01-01T00:00:00Z')"
756     )
757     await conn.execute("INSERT INTO filter_config (user_id, config_json) VALUES
758 (111, '{}')")
759     await conn.commit()
760     # user_version is 0 on a freshly created DB.
761
762 # Now open via init_db ? migrations 001, 002, and 003 must run.
763 await db.init_db(db_path)
764
765 async with aiosqlite.connect(db_path) as conn:
766     await conn.execute("PRAGMA foreign_keys=ON")
767     # Column added (migration 001) ? on the legacy videos table, now renamed.
768     assert await db._column_exists(conn, "videos_deprecated", "document_id")
769     # Identity split (migration 002): users renamed, accounts present.
770     assert await db._table_exists(conn, "accounts")
771     assert await db._table_exists(conn, "telegram_sessions")
772     assert await db._table_exists(conn, "users_deprecated")
773     assert not await db._table_exists(conn, "users")
774     # accounts.id copied from users.id (integer-preserving).
775     async with conn.execute("SELECT id, telegram_id, provider FROM accounts") as
776 cur:
777         rows = await cur.fetchall()
778         assert rows == [(111, 111, "telegram")]
779         # telegram_sessions row derived from the legacy user row.
780         async with conn.execute(
781             "SELECT account_id, phone, session_path, enabled FROM telegram_sessions"
782         ) as cur:
783             srows = await cur.fetchall()
784             assert srows == [(111, "+1", "user_111.session", 1)]

```



```

780
781     # Domain schema (migration 003): legacy tables renamed to *_deprecated,
782     # provider-neutral tables backfilled from them.
783     assert await db._table_exists(conn, "sources")
784     assert await db._table_exists(conn, "content_items")
785     assert await db._table_exists(conn, "policies")
786     assert await db._table_exists(conn, "channels_deprecated")
787     assert await db._table_exists(conn, "videos_deprecated")
788     assert await db._table_exists(conn, "filter_config_deprecated")
789     assert not await db._table_exists(conn, "channels")
790     assert not await db._table_exists(conn, "videos")
791     assert not await db._table_exists(conn, "filter_config")
792
793     # FK contract (PR #33 review): the rebuilt legacy tables (now
794     # *_deprecated) reference accounts(id), NOT users_deprecated(id).
795     for tbl in ("channels_deprecated", "videos_deprecated",
796 "filter_config_deprecated"):
797         async with conn.execute(f"PRAGMA foreign_key_list({tbl})") as cur:
798             fk_rows = await cur.fetchall()
799             referenced_tables = {r[2] for r in fk_rows}
800             assert "accounts" in referenced_tables, f"{tbl} FK does not reference
accounts"
801             assert (
802                 "users_deprecated" not in referenced_tables
803                 ), f"{tbl} FK still references users_deprecated (rename-rewrite bug)"
804
805     # Legacy data preserved in the deprecated tables.
806     async with conn.execute(
807         "SELECT message_id, document_id, user_id FROM videos_deprecated"
808     ) as cur:
809         vrows = await cur.fetchall()
810         assert vrows == [(1, None, 111)]
811     async with conn.execute(
812         "SELECT user_id, channel_id, title FROM channels_deprecated"
813     ) as cur:
814         crows = await cur.fetchall()
815         assert crows == [(111, -1001, "pre-migration")]
816
817     # Domain backfill (migration 003): sources + content_items populated.
818     async with conn.execute(
819         "SELECT account_id, provider, provider_source_id, title, watched " "FROM
sources"
820     ) as cur:
821         src_rows = await cur.fetchall()
822         assert src_rows == [(111, "telegram", "-1001", "pre-migration", 1)]
823     async with conn.execute(
824         "SELECT account_id, source_id, provider_message_id, document_id, "
825         "duration_s, width, height, size_bytes, dedup_key FROM content_items"
826     ) as cur:
827         ci_rows = await cur.fetchall()
828         # dedup_key falls to the Tier-0 branch (no document_id, but the
829         # fingerprint fields are all present) ? actually the fingerprint branch
830         # wins here since size/duration/h/w are all set.
831         assert len(ci_rows) == 1
832         ci = ci_rows[0]
833         assert ci[0] == 111 # account_id
834         assert ci[2] == "1" # provider_message_id (cast to TEXT)
835         assert ci[3] is None # document_id
836         assert ci[4:8] == (60, 1920, 1080, 1000000) # dur, w, h, size
837         assert ci[8] == "tg:fp:1000000:60:1080:1920" # fingerprint dedup_key
838
839     # Policy backfill (migration 003): one default policy per account.
840     async with conn.execute(
841         "SELECT account_id, name, config_json, is_default FROM policies"
842     ) as cur:

```

```

842         pol_rows = await cur.fetchall()
843         assert pol_rows == [(111, "default", "{}", 1)]
844
845         # user_version bumped to the highest registered migration.
846         async with conn.execute("PRAGMA user_version") as cur:
847             assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
848
849
850     async def test_migration_002_retargeted_fks_allow_inserts_for_new_accounts(
851         tmp_path: object,
852     ) -> None:
853         """Regression guard for PR #33: post-migration, a channel for a brand-new
854         account (not present in users_deprecated) must insert cleanly under

```

22. assistant (2026-07-06T23:49:42.610Z)

Now I have everything to write a focused repro. Let me write a script that: (1) migrates a legacy v1 DB, checks policy_decisions structure + FK targets + user_version=5; (2) tests FK enforcement (inserting a policy_decision with bad content_item_id/policy_id should fail); (3) tests that _SCHEMA-created and migration-005-created policy_decisions are structurally identical; (4) fresh double-init idempotency.

23. assistant (2026-07-06T23:50:07.588Z)

```

import asyncio, tempfile, os
import aiosqlite
from tg_video_filter import db

LEGACY = """
CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL DEFAULT 1,
created_at TEXT NOT NULL);
CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL REFERENCES
users(id),
    channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL, UNIQUE (user_id,
channel_id));
CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL REFERENCES
users(id),
    channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, duration_s INTEGER, width INTEGER,
height INTEGER,
    size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0, forwarded INTEGER
NOT NULL DEFAULT 0,
    seen_at TEXT NOT NULL, UNIQUE (user_id, channel_id, message_id));
CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT
NOT NULL);
"""

def norm(sql):
    # crude normalization of a CREATE TABLE sql for comparison
    import re
    return re.sub(r"\s+", " ", sql.strip().lower())

async def get_create_sql(conn, table):
    async with conn.execute("SELECT sql FROM sqlite_master WHERE type='table' AND name=?",
(table,)) as c:
        r = await c.fetchone()
        return r[0] if r else None

async def main():
    d = tempfile.mkdtemp()

    # ---- Legacy v1 path ----
    legacy = os.path.join(d, "legacy.db")
    async with aiosqlite.connect(legacy) as conn:
        await conn.executescript(LEGACY)
        await conn.execute("INSERT INTO users (id,phone,enabled,created_at) VALUES

```

```

(111, '+1', 1, '2024-01-01T00:00:00Z'))
    await conn.execute("INSERT INTO channels (user_id,channel_id,title,seen_first_at) VALUES
(111,-1001,'c','2024-01-01T00:00:00Z')")
    await conn.execute("INSERT INTO videos (user_id,channel_id,message_id,duration_s,width,height,size_bytes,mime_type,matched,forwarded,seen_at) VALUES
(111,-1001,1,60,1920,1080,1000000,'video/mp4',1,1,'2024-01-01T00:00:00Z')")
    await conn.execute("INSERT INTO filter_config (user_id,config_json) VALUES (111, '{}')")
    await conn.commit()

await db.init_db(legacy)
legacy_create = None
async with aiosqlite.connect(legacy) as conn:
    await conn.execute("PRAGMA foreign_keys=ON")
    async with conn.execute("PRAGMA user_version") as c:
        uv = (await c.fetchone())[0]
        print("LEGACY user_version =", uv, "(expect 5)")
        legacy_create = await get_create_sql(conn, "policy_decisions")
        print("LEGACY policy_decisions exists:", legacy_create is not None)
        async with conn.execute("PRAGMA foreign_key_list(policy_decisions)") as c:
            fks = await c.fetchall()
            print("LEGACY policy_decisions FKs ->", {r[2] for r in fks})
            # FK enforcement: insert good then bad
            # good: content_item id 1 + policy id 1 exist from backfill
            async with conn.execute("SELECT id FROM content_items") as c:
                ci = await c.fetchall()
            async with conn.execute("SELECT id FROM policies") as c:
                pol = await c.fetchall()
            print("content_items ids:", ci, "policies ids:", pol)
            # bad FK: content_item_id=9999 should FAIL with foreign_keys ON
            try:
                await conn.execute("INSERT INTO policy_decisions
(content_item_id,policy_id,passed,trace_json,evaluated_at) VALUES (9999,?,1,[''],'t')",
(pol[0][0],))
                await conn.commit()
                print("FK ENFORCEMENT (bad content_item): NO ERROR <-- BUG if this prints")
            except Exception as e:
                print("FK ENFORCEMENT (bad content_item): raised", type(e).__name__)
            # bad policy_id
            try:
                await conn.execute("INSERT INTO policy_decisions
(content_item_id,policy_id,passed,trace_json,evaluated_at) VALUES (?,9999,1,[''],'t')",
(ci[0][0],))
                await conn.commit()
                print("FK ENFORCEMENT (bad policy): NO ERROR <-- BUG if this prints")
            except Exception as e:
                print("FK ENFORCEMENT (bad policy): raised", type(e).__name__)

# ---- Fresh path ----
fresh = os.path.join(d, "fresh.db")
await db.init_db(fresh)
await db.init_db(fresh) # double init idempotency
async with aiosqlite.connect(fresh) as conn:
    async with conn.execute("PRAGMA user_version") as c:
        print("FRESH user_version =", (await c.fetchone())[0], "(expect 5)")
        fresh_create = await get_create_sql(conn, "policy_decisions")
        async with conn.execute("SELECT count(*) FROM sqlite_master WHERE
name='policy_decisions'") as c:
            print("FRESH policy_decisions table count:", (await c.fetchone())[0])

print("\nSCHEMA MATCH (fresh vs legacy-migrated):", norm(fresh_create) ==
norm(legacy_create))
if norm(fresh_create) != norm(legacy_create):
    print("FRESH :", norm(fresh_create))
    print("LEGACY:", norm(legacy_create))

```

```

# ---- Re-run migration on a DB that already has policy_decisions (partial prior run sim)
----
# Simulate a DB at version 4 that somehow already has policy_decisions (IF NOT EXISTS
safety)
partial = os.path.join(d, "partial.db")
await db.init_db(partial)
async with aiosqlite.connect(partial) as conn:
    await conn.execute("PRAGMA user_version = 4") # pretend mig 005 hasn't been marked
    await conn.commit()
await db.init_db(partial) # should re-run mig 005 harmlessly (IF NOT EXISTS)
async with aiosqlite.connect(partial) as conn:
    async with conn.execute("PRAGMA user_version") as c:
        print("\nPARTIAL re-run user_version =", (await c.fetchone())[0], "(expect 5, no
crash)")

asyncio.run(main())

```

24. user (2026-07-06T23:50:07.589Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

25. user (2026-07-06T23:50:32.401Z)

```

LEGACY user_version = 5 (expect 5)
LEGACY policy_decisions exists: True
LEGACY policy_decisions FKs -> ['content_items', 'policies']
content_items ids: [(1,)] policies ids: [(1,)]
FK (bad content_item): raised IntegrityError
FK (bad policy): raised IntegrityError
FRESH user_version = 5 (expect 5)
SCHEMA MATCH (fresh vs legacy-migrated): True
PARTIAL re-run user_version = 5 (expect 5)

```

26. assistant (2026-07-06T23:50:40.881Z)

All core migration/schema invariants hold:

- Legacy v1 ? user_version=5, policy_decisions created, FKs correctly point to content_items + policies.
- FK enforcement works (bad content_item_id and bad policy_id both raise IntegrityError).
- Fresh (_SCHEMA) and legacy-migrated (mig 005) schemas are byte-identical after normalization.
- Partial re-run (version reset to 4) re-runs mig 005 harmlessly via IF NOT EXISTS, reaches 5.

Now let me probe the subtler concern flagged in the prompt: the **drop-empty-legacy logic** in `_apply_schema`. On a legacy DB, `_apply_schema` runs `executescript(_SCHEMA)` which creates `policy_decisions`. Then IF `channels_deprecated` exists it drops `channels/videos/filter_config`. This runs on EVERY init. The concern: does a **second** init on an already-migrated legacy DB behave correctly, and does the WAL/foreign_keys state affect anything? Let me also test the specific scenario where FK enforcement is critical during migration 002's table rebuilds ? does the pre-existing policy_decisions (empty) survive migration 002/003's operations without FK violations, and does re-init on a migrated DB keep policy_decisions intact with data.

27. user (2026-07-06T23:51:00.076Z)

```

after record: rows = 1
after re-init: policy_decisions rows = 1 (expect 1 - data must survive)
after re-init: user_version = 5
row content: [(1, 1, 1)]
after re-record: rows = 1 passed = 0 (expect 1 row, passed=0)

```

28. assistant (2026-07-06T23:51:03.325Z)

Data survives re-init and UPSERT is idempotent. Let me check the `record_policy_decision` signature and one edge: the UNIQUE constraint. Let me read the repo functions to check for any write-correctness issue in the diff.

29. user (2026-07-06T23:51:04.438Z)

```
1396 async def record_policy_decision(
1397     conn: aiosqlite.Connection,
1398     *,
1399     content_item_id: int,
1400     policy_id: int,
1401     passed: bool,
1402     trace: list[dict[str, object]],
1403     evaluated_at: datetime | None = None,
1404 ) -> PolicyDecision:
1405     """Persist or replace the policy evaluation trace for a content item."""
1406     evaluated_at = evaluated_at or _now()
1407     trace_json = _policy_trace_json(trace)
1408     await conn.execute(
1409         """INSERT INTO policy_decisions
1410             (content_item_id, policy_id, passed, trace_json, evaluated_at)
1411             VALUES (?, ?, ?, ?, ?)
1412             ON CONFLICT(content_item_id, policy_id) DO UPDATE SET
1413                 passed=excluded.passed,
1414                 trace_json=excluded.trace_json,
1415                 evaluated_at=excluded.evaluated_at""",
1416         (
1417             content_item_id,
1418             policy_id,
1419             int(passed),
1420             trace_json,
1421             evaluated_at.isoformat(),
1422         ),
1423     )
1424     await conn.commit()
1425     decision = await get_policy_decision(conn, content_item_id, policy_id)
1426     assert decision is not None
1427     return decision
1428
1429
1430 async def get_policy_decision(
1431     conn: aiosqlite.Connection,
1432     content_item_id: int,
1433     policy_id: int,
1434 ) -> PolicyDecision | None:
1435     """Read a policy decision by its natural key."""
1436     conn.row_factory = aiosqlite.Row
1437     async with conn.execute(
1438         """SELECT * FROM policy_decisions
1439             WHERE content_item_id = ? AND policy_id = ?""",
1440         (content_item_id, policy_id),
1441     ) as cur:
1442         row = await cur.fetchone()
1443         return _row_to_policy_decision(row) if row else None
1444
1445
1446 async def list_policy_decisions_for_content(
1447     conn: aiosqlite.Connection,
1448     content_item_id: int,
1449 ) -> list[PolicyDecision]:
1450     """Return all policy decisions recorded for a content item."""
1451     conn.row_factory = aiosqlite.Row
1452     async with conn.execute(
1453         """SELECT * FROM policy_decisions
1454             WHERE content_item_id = ?
1455             ORDER BY evaluated_at, id""",
1456         (content_item_id, ),
1457     ) as cur:
1458         rows = await cur.fetchall()
```

```
1459         return [_row_to_policy_decision(row) for row in rows]
1460
```

30. assistant (2026-07-06T23:51:10.618Z)

Everything checks out for the migration/schema dimension. Let me confirm one last thing: whether ``record_policy_decision`` runs with ``foreign_keys=ON`` in production. The FK is only enforced if the connection has ``PRAGMA foreign_keys=ON``. ``_apply_schema`` sets it, but that's a per-connection pragma ? does the production connection used at write time have it enabled? Let me check how the app connection is opened.

31. user (2026-07-06T23:51:14.233Z)

```
9:in-memory database (`aiosqlite.connect(":memory:")`).
210:     async with aiosqlite.connect(db_path) as conn:
218:         await conn.execute("PRAGMA foreign_keys = ON")
=== telethon_client connection ===
342:         await _record_policy_decision(
466:async def _record_policy_decision(
477:     await db.record_policy_decision(
520:         "_record_policy_decision",
```

32. assistant (2026-07-06T23:51:17.278Z)

``init_db`` (line 210) opens its own connection, sets ``foreign_keys=ON``, applies schema, and closes it. The connection used at `*write*` time (in `telethon_client`) is separate. Let me check how that connection is opened and whether it sets ``foreign_keys=ON``.

33. user (2026-07-06T23:51:19.237Z)

(Bash completed with no output)

34. user (2026-07-06T23:51:22.053Z)

```
466     async def _record_policy_decision(
467         conn: aiosqlite.Connection,
468         *,
469         inserted: VideoRecord,
470         policy_id: int | None,
471         policy_passed: bool,
472         policy_results: list[StageResult],
473     ) -> None:
474         """Persist a policy trace after the content item has been inserted."""
475         if not policy_results or policy_id is None or inserted.id is None:
476             return
477         await db.record_policy_decision(
478             conn,
479             content_item_id=inserted.id,
480             policy_id=policy_id,
481             passed=policy_passed,
482             trace=stage_results_to_trace(policy_results),
483         )
484
485
486     def _peer_id(peer: object) -> int:
487         """Best-effort numeric id for the peer of a message.
488
489         Telegram represents channel/group/user peers as different TL types; we
490         collapse them to a single int for dedup/storage. The exact subclass isn't
491         important ? only that the id is stable per chat.
492         """
493         # Each Peer* type has a field like channel_id / chat_id / user_id.
494         for field in ("channel_id", "chat_id", "user_id"):
495             val = getattr(peer, field, None)
```

```

496         if val is not None:
497             return int(val)
498         # Fallback: hash the repr so we never collide with real ids (negative).
499         return -abs(hash(repr(peer)) % (2**31))
500
501
502     def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
503         """Best-effort channel title from the event, if available."""
504         chat = getattr(event, "chat", None)
505         return getattr(chat, "title", None) if chat is not None else None
506
507
508     class _NewMessageCallback:
509         """Structural type annotation for the handler returned by make*_handler."""
510
511         async def __call__(self, event: NewMessage.Event) -> None: ...
512
513
514     __all__ = [
515         "build_client",
516         "extract_video_meta",
517         "make_new_message_handler",
518         "_content_item_preview",
519         "_evaluate_policy_stages",
520         "_record_policy_decision",

```

35. assistant (2026-07-06T23:51:27.053Z)

The `conn` is passed in. Let me trace where that connection originates to confirm whether `foreign\_keys=ON` is set on it. This matters: if the write-time connection has FK OFF (SQLite default is OFF per-connection), then a bad content_item_id/policy_id would NOT be caught ? but the guard in `\_record\_policy\_decision` and the fact `inserted.id` comes from a just-inserted row makes a dangling FK unlikely in practice. Still, worth confirming the connection setup ? but note this is about the *caller's* connection setup which is largely pre-existing code, not this PR's diff. Let me check where the handler's conn is created.

36. user (2026-07-06T23:51:29.297Z)

```

C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\backfill.py:103:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\backfill.py:340: def
make_backfill_command_handler( # pragma: no cover - requires live Telethon connection
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\backfill.py:343:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\backfill.py:436:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\backfill.py:470: __all__ = ["backfill_user",
"make_backfill_command_handler"]
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\commands.py:55:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\commands.py:95:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\bootstrap.py:63:    async with
aiosqlite.connect(db_path) as conn:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\delivery.py:127:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\delivery.py:166:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\delivery.py:198:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\command_service.py:211:    conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\command_service.py:348:    self, conn:
aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\command_service.py:364:    self, conn:

```

```

aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\command_service.py:376:    async def
_handle_channels(self, conn: aiosqlite.Connection, user_id: int) -> CommandResult:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\command_service.py:393:    async def
_handle_watchlist(self, conn: aiosqlite.Connection, user_id: int) -> CommandResult:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\command_service.py:409:        self, conn:
aiosqlite.Connection, user_id: int, target: str
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\command_service.py:437:        conn:
aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:9:in-memory database
(``aiosqlite.connect(":memory:")``).
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:210:    async with
aiosqlite.connect(db_path) as conn:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:214:async def _apply_schema(conn:
aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:218:    await conn.execute("PRAGMA
foreign_keys = ON")
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:330:async def _apply_migrations(conn:
aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:343:async def _column_exists(conn:
aiosqlite.Connection, table: str, column: str) -> bool:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:351:async def
_migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:368:async def
_migration_002_identity_split(conn: aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:520:async def
_migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:683:async def
_migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:779:async def
_migration_005_policy_decisions(conn: aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:796:async def _table_exists(conn:
aiosqlite.Connection, table: str) -> bool:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:983:async def upsert_account(conn:
aiosqlite.Connection, account: Account) -> Account:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1004:async def get_account(conn:
aiosqlite.Connection, account_id: int) -> Account | None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1012:    conn: aiosqlite.Connection,
telegram_id: int
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1022:    conn: aiosqlite.Connection,
session: TelegramSession
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1057:    conn: aiosqlite.Connection,
account_id: int
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1068:    conn: aiosqlite.Connection,
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1105:async def upsert_user(conn:
aiosqlite.Connection, user: User) -> User:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1128:async def get_user(conn:
aiosqlite.Connection, user_id: int) -> User | None:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1137:async def
list_enabled_users(conn: aiosqlite.Connection) -> list[User]:
C:\Users\avidu\Projects\wt-pr55\src\tg_video_filter\db.py:1151:async def upsert_source(conn:
aiosqlite.Connection, source: Source) -> Source:

```

[Showing results with pagination = limit: 40]

37. assistant (2026-07-06T23:51:35.639Z)

The runtime connection is opened in bootstrap.py:63. This is pre-existing infrastructure ? not this PR's diff ? and the FK-ON status of the runtime connection affects ALL FK tables equally, not something F6 introduced. The `\_record\_policy\_decision` guard plus the fact `inserted.id` is a real just-inserted content_item makes a dangling FK effectively impossible in the actual call path regardless.

My migration/schema-dimension investigation is complete. All invariants verified via repro. No defects found in the migration 005 + schema correctness dimension. The known deviations (per-

decision-JSON vs per-stage-rows, no TTL, metadata-rejected items get no row) are ADR-level design choices already flagged by the main reviewer, not defects in this dimension.

38. user (2026-07-06T23:51:35.644Z)

Structured output provided successfully